# Assignment 6.3

**2303a51154**

**Bt:17**

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.

• The class should include attributes such as name, roll number, and branch.

• Add a method display_details() to print student information.

• Execute the code and verify the output.

• Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

• A Python class with a constructor (__init__) and a display_details() method.

• Sample object creation and output displayed on the console.

• Brief analysis of AI-generated code.

## #code:

```python
#Create a Python program that defines a Student class.
#The class should include the following attributes:
#name
#roll_number
#branch
#Add a method display_details() that prints the student’s information in a readable format.
#Write code to:
#Create at least one Student object
#all the display_details() method
#Execute the program and show the output
#After generating the code, analyze it for correctness, readability, and proper use of class concepts
# .
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")
# Example usage
student1 = Student("Alice Smith", "101", "Computer Science")
student1.display_details()
student2 = Student("Bob Johnson", "102", "Mechanical Engineering")
student2.display_details()
student3 = Student("Charlie Brown", "103", "Electrical Engineering")
student3.display_details()
student4 = Student("Diana Prince", "104", "Civil Engineering")
student4.display_details()
student5 = Student("Ethan Hunt", "105", "Chemical Engineering")
student5.display_details()
student6 = Student("Fiona Gallagher", "106", "Biotechnology")
student6.display_details()
student7 = Student("George Martin", "107", "Aerospace Engineering")
student7.display_details()
student8 = Student("Hannah Baker", "108", "Environmental Science")
```

**#Output**

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\HP> & C:/Users/HP/AppData/Local/Microsoft/WindowsApps/python3.13.exe c:/Users/HP/Downloads/Lab6.3.py
Student Name: Alice Smith
Roll Number: 101
Branch: Computer Science
Student Name: Bob Johnson
Roll Number: 102
Branch: Mechanical Engineering
Student Name: Charlie Brown
Roll Number: 103
Branch: Electrical Engineering
Student Name: Diana Prince
Roll Number: 104
Branch: Civil Engineering
Student Name: Ethan Hunt
Roll Number: 105
Branch: Chemical Engineering
Student Name: Fiona Gallagher
Roll Number: 106
Branch: Biotechnology
Student Name: George Martin
Roll Number: 107
Branch: Aerospace Engineering
Student Name: Hannah Baker
Roll Number: 108
Branch: Environmental Science
```

**Task Description #2: Loops (Multiples of a Number)**

**Scenario**

**You are writing a utility function to display multiples of a given number.**

**Task**

**• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number**

**using a loop.**

**• Analyze the generated loop logic.**

**• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,**

**while instead of for).**

**Expected Output #2**

**• Correct loop-based Python implementation.**

**• Output showing the first 10 multiples of a number.**

• **Comparison and analysis of different looping approaches.**

**#code:**

```
#Scenario
#You are writing a utility function to display multiples of a given number.
#Task
#• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number
#using a loop.
#• Analyze the generated loop logic.
#• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,
#while instead of for).
student8.display_details()
def print_multiples(number):
    for i in range(1, 11):
        print(number * i)
# Example usage
print("Multiples of 5:")
print_multiples(5)
def print_multiples_while(number):
    count = 1
    while count <= 10:
        print(number * count)
        count += 1
# Example usage
print("Multiples of 7:")
print_multiples_while(7)
```

**#output**

```
Multiples of 5:
5
10
15
20
25
30
35
40
45
50
Multiples of 7:
7
14
21
28
35
42
49
56
63
70
```

**Task Description #3: Conditional Statements (Age Classification)**

**Scenario**

**You are building a basic classification system based on age.**

**Task**

**• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups**

**(e.g., child, teenager, adult, senior).**

**• Analyze the generated conditions and logic.**

**• Ask the AI to generate the same classification using alternative conditional structures (e.g.,**

**simplified conditions or dictionary-based logic).**

**Expected Output #3**

- **A Python function that classifies age into appropriate groups.**
- **Clear and correct conditional logic.**
- **Explanation of how the conditions work.**

**#code:**

```python
#Scenario
#You are building a basic classification system based on age.
#Task
#• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups
#(e.g., child, teenager, adult, senior).
#• Analyze the generated conditions and logic.
#• Ask the AI to generate the same classification using alternative conditional structures (e.g.,
#simplified conditions or dictionary-based logic).
def classify_age(age):
    if age < 0:
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teenager"
    elif age <= 59:
        return "Adult"
    else:
        return "Senior"
# Example usage
ages = [5, 15, 25, 65, -3]
for age in ages:
    print(f"Age: {age}, Classification: {classify_age(age)}")
def classify_age_dict(age):
    age_groups = {
        "Child": range(0, 13),
        "Teenager": range(13, 20),
        "Adult": range(20, 60),
        "Senior": range(60, 150)
    }
    if age < 0:
        return "Invalid age"
    for group, age_range in age_groups.items():
        if age in age_range:
            return group
    return "Invalid age"
# Example usage
for age in ages:
    print(f"Age: {age}, Classification: {classify_age_dict(age)}")
```

**#output**

```
Age: 5, Classification: Child
Age: 15, Classification: Teenager
Age: 25, Classification: Adult
Age: 65, Classification: Senior
Age: -3, Classification: Invalid age
Age: 5, Classification: Child
Age: 15, Classification: Teenager
Age: 25, Classification: Adult
Age: 65, Classification: Senior
Age: -3, Classification: Invalid age
```

**Task Description #4: For and While Loops (Sum of First n Numbers)**

**Scenario**

**You need to calculate the sum of the first n natural numbers.**

**Task**

- **Use AI assistance to generate a sum_to_n() function using a for loop.**
- **Analyze the generated code.**
- **Ask the AI to suggest an alternative implementation using a while loop or a mathematical**

**formula.**

**Expected Output #4**

- **Python function to compute the sum of first n numbers.**
- **Correct output for sample inputs.**
- **Explanation and comparison of different approaches.**

**#code**

```python
#Scenario
#You need to calculate the sum of the first n natural numbers.
#Task
#• Use AI assistance to generate a sum_to_n() function using a for loop.
#• Analyze the generated code.
#• Ask the AI to suggest an alternative implementation using a while loop or a mathematical
#formula.
def sum_to_n_for(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
# Example usage
n = 10
print(f"Sum of first {n} natural numbers using for loop: {sum_to_n_for(n)}")
def sum_to_n_while(n):
    total = 0
    count = 1
    while count <= n:
        total += count
        count += 1
    return total
# Example usage
print(f"Sum of first {n} natural numbers using while loop: {sum_to_n_while(n)}")
def sum_to_n_formula(n):
    return n * (n + 1) // 2
# Example usage
print(f"Sum of first {n} natural numbers using formula: {sum_to_n_formula(n)}")
```

**#output**

```
Sum of first 10 natural numbers using for loop: 55
Sum of first 10 natural numbers using while loop: 55
Sum of first 10 natural numbers using formula: 55
```

**Task Description #5: Classes (Bank Account Class)**

**Scenario**

**You are designing a basic banking application.**

**Task**

**• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),**

**and check_balance().**

**• Analyze the AI-generated class structure and logic.**

**• Add meaningful comments and explain the working of the code.**

**Expected Output #5**

**• Complete Python Bank Account class.**

**• Demonstration of deposit and withdrawal operations with updated balance.**

**• Well-commented code with a clear explanation.**

## #code

```python
#Scenario
#You are designing a basic banking application.
#Task
#• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),
#and check_balance().
#• Analyze the AI-generated class structure and logic.
#• Add meaningful comments and explain the working of the code.

class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}. New balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        elif amount <= 0:
            print("Withdrawal amount must be positive.")
        else:
            self.balance -= amount
            print(f"Withdrew: {amount}. New balance: {self.balance}")

    def display_balance(self):
        print(f"Account holder: {self.account_holder}, Balance: {self.balance}")
# Example usage
account = BankAccount("Alice", 1000)
account.display_balance()
account.deposit(500)
account.withdraw(200)
account.withdraw(2000)
account.display_balance()
```

## #output:

```
Account holder: Alice, Balance: 1000
Deposited: 500. New balance: 1500
Withdrew: 200. New balance: 1300
Insufficient funds.
Account holder: Alice, Balance: 1300
PS C:\Users\HP>
```